# SecureVault

Academic project documentation

# Contents

Project Synopsis

Abstract

Problem statement

Aim

Objectives

Scope and operating limits

Existing baseline and proposed solution

Users and modules

Functional requirements

Nonfunctional requirements

Technical design

Secure software development life cycle

Verification approach and current result

Feasibility and limitations

References

# Project Synopsis

• Student: [Student name]

• Enrollment number: [Enrollment number]

• Programme and semester: [Programme and semester]

• Department: [Department]

• Institution: [College or university]

• Project guide: [Guide name and designation]

• Academic year: [Academic year]

# Abstract

SecureVault is a cybersecurity web application for accepting, inspecting and privately storing user files. It treats filenames and browser-declared content types as untrusted, validates supported content within explicit resource limits, applies local inspection policies and reconstructs selected formats into reduced-feature representations. Accepted files are stored as encrypted blobs outside the public web directory. Ownership checks and integrity verification protect retrieval, while optional time-based multi-factor authentication and expiring single-use share codes demonstrate account and access controls.

The implementation uses Python 3.12, Flask, Waitress and SQLite. Security layers include scrypt password hashing, CSRF protection, session revocation, rate limits, format-specific validation, local ClamAV antivirus, YARA-X rules, pattern-based DLP and an operator-managed SHA-256 denylist. Native Tesseract OCR can provide text from image and PDF inputs to the DLP checks. OpenCV optionally detects and corrects document-photo perspective. These controls have distinct roles and limitations; the project does not claim universal malware detection, commercial Deep CDR, multiple commercial antivirus engines or dynamic sandbox analysis. The work follows an iterative secure software development life cycle with requirements, threat-informed design, implementation, verification and maintenance activities.

# Problem statement

An unrestricted file-upload feature can accept executable content disguised by a filename, malformed parser inputs, oversized requests or documents containing active content and unintended metadata. A filename extension and a browser-provided MIME type are only client claims. A file-management service also needs access control, private storage, integrity checks and a deliberate sharing model. Educational implementations must demonstrate both what their controls do and what they cannot guarantee.

SecureVault addresses these concerns through a bounded allowlist, independent content checks, format-specific reconstruction, local detection layers, encrypted storage and owner-controlled retrieval. Its supported inputs are TXT, PNG, JPEG, PDF, DOCX, CSV, HTML, XML and restricted SVG. Generic archives, executables, legacy Office binaries and arbitrary formats are rejected.

## Aim

To design, implement and evaluate an explainable secure file-upload workflow that demonstrates layered input validation, privacy controls, local inspection, encrypted storage and controlled sharing.

## Objectives

• Authenticate users and enforce ownership on file operations.

• Check filename, MIME, content signatures where defined, parser structure and resource bounds.

• Re-encode supported images to remove embedded metadata.

• Reconstruct selected document formats to reduce active content and preserve only documented safe content.

• Apply local YARA-X and ClamAV checks, with uploads rejected if required antivirus scanning is unavailable or fails.

• Use OCR-assisted text checks for eligible image/PDF files when Tesseract is installed.

• Encrypt accepted file content at rest and verify its integrity before access.

• Provide optional TOTP multi-factor authentication, revocable sessions and a tamper-evident activity chain.

• Permit deliberate sharing through hashed, expiring, single-use codes with owner revocation.

• Verify implementation behavior with automated regression cases and a synthetic-data demonstration.

## Scope and operating limits

| Limit or capability | Implemented value |
|---|---|
| Maximum upload size | 50 MiB per file |
| Per-account storage | 500,000,000 bytes (500 MB decimal) |
| Per-account file count | 200 files |
| Self-registration | Maximum five accounts by default; configurable |
| Image bounds | Decoder checks and a 16-megapixel limit |
| PDF bounds | Up to ten pages for bounded reconstruction/OCR paths |
| Share code | 96 random bits, 15-minute expiry, one successful redemption |
| Storage | Private encrypted file blobs; SQLite metadata and access records |

The stored object may differ from the uploaded original. PDF reconstruction removes searchable text, signatures, links, forms, attachments and original metadata. DOCX reconstruction preserves plain paragraphs but not images, formatting, macros, fields or embedded objects. CSV formula-like values are prefixed with an apostrophe. HTML, XML and SVG are rebuilt under restrictive policies. Users should retain an original copy when those properties matter.

## Existing baseline and proposed solution

The following comparison describes a basic educational upload design, not a measured comparison with a named product.

| Concern | Basic upload baseline | SecureVault approach |
|---|---|---|
| File identity | Trust extension or MIME | Compare claims with bytes and parser behavior |
| Input size | No enforced boundary | Request, file, parser, page and image bounds |
| Stored content | Save original under user filename | Reconstruct/normalize and assign random internal ID |
| Metadata | Preserve image metadata | Re-encode supported images; remove embedded metadata |
| Access | Predictable direct path | Authenticated owner checks on each file action |
| Storage privacy | Public or plaintext folder | Fernet-encrypted blobs in a private data directory |
| Detection | No local threat indicators | ClamAV, local YARA-X rules and optional SHA-256 denylist |
| Image documents | Upload photo as-is | Optional OpenCV page detection and perspective correction |
| Scanned text | Not inspected | Local Tesseract OCR can feed text into DLP |
| Sharing | Public link or direct path | Hashed, expiring, revocable and single-use code |

## Users and modules

The primary user is an authenticated owner uploading synthetic or non-sensitive documents for a small file-vault demonstration. A recipient may redeem a share code when the owner’s inspection policy allows sharing. The operator installs and maintains local inspection tools; the operator is outside the application’s cryptographic trust boundary because the server holds the file keys.

| Module | Responsibility |
| --- | --- |
| Account and session management | Registration, sign-in, logout, optional TOTP and session revocation |
| Upload validation | Safe names, MIME agreement, signatures, decoding and limits |
| Content processing | Image normalization, OpenCV option and bounded document reconstruction |
| Security inspection | ClamAV, YARA-X, hash policy, OCR and text DLP outcomes |
| Encrypted storage | Random blob identifiers, authenticated encryption and integrity verification |
| Controlled sharing | Issue, expire, revoke and atomically redeem one-time codes |
| Audit and reporting | HMAC-linked events and owner-only per-file inspection report |
| User interface | Landing page, vault, upload progress, file actions and report views |

## Functional requirements

| ID | Requirement | Acceptance condition |
|---|---|---|
| FR01 | Register and authenticate | Valid account can sign in; invalid credentials fail safely; account cap is enforced |
| FR02 | Upload supported content | Valid allowlisted input passes the defined checks and policy |
| FR03 | Reject invalid content | Unsupported, malformed, mismatched or over-limit input is not stored |
| FR04 | Inspect uploaded files | Reports distinguish validation, antivirus, YARA-X, OCR and DLP outcomes |
| FR05 | Normalize and reconstruct | Image metadata and active document features are removed according to format policy |
| FR06 | Store and retrieve privately | Accepted bytes are encrypted; retrieval requires ownership and valid integrity |
| FR07 | Manage account security | Logout, revocation and optional TOTP replay protection work as documented |
| FR08 | Record user activity | Owner sees their activity and an explicit HMAC-chain status |
| FR09 | Share a selected file | Only a valid, permitted, unused and unexpired code allows one redemption |

## Nonfunctional requirements

The application must use parameterized database operations, keep file bytes outside a public static directory, apply bounded processing and avoid logging raw file contents or matched sensitive values. Forms should have visible labels, keyboard-operable controls, understandable errors and reduced-motion behavior. Per-file and per-account limits must be enforced server-side. Security failures must not be represented as successful checks. Private runtime data and generated keys must remain outside the source repository.

## Technical design

| Area | Technology and role |
| --- | --- |
| Application | Python 3.12, Flask 3.1 and Waitress |
| Interface | Jinja templates, HTML, CSS and JavaScript |
| Database | SQLite with parameterized queries and foreign-key enforcement |
| Image/document processing | Pillow, OpenCV, PDFium, defusedxml and nh3 |
| Authentication | Werkzeug scrypt, Flask-WTF, Flask-Limiter and PyOTP |
| Encryption and integrity | Fernet, SHA-256 and HMAC-SHA-256 |
| Local detection | ClamAV and YARA-X; native Tesseract for optional OCR |
| Verification | pytest, Bandit, pip-audit and GitHub Actions |

Files are processed synchronously. A short-lived reconstruction process applies deadlines and bounds but is not an operating-system sandbox. ClamAV is required by default and its local signatures must be maintained. YARA-X signatures are educational local indicators, not a complete threat feed. OCR is subject to language, image-quality and recognition limits. Fernet is server-side encryption at rest and is not end-to-end encryption.

## Secure software development life cycle

The project uses an iterative secure SDLC: each phase includes security tasks and produces a reviewable artifact. Phase completion depends on evidence, not elapsed weeks.

| Phase | Activities | Deliverable / review point |
| --- | --- | --- |
| Planning | Define the upload risk, users, scope and constraints | Synopsis and scope boundaries |
| Requirements | Specify functional, security and usability conditions | Requirement tables and acceptance criteria |
| Analysis and design | Identify trust boundaries, misuse cases, data flow and schema | Architecture, DFD, ER diagram and threat analysis |
| Implementation | Build the app in modules and keep controls server-side | Source code, reviewable configuration and synthetic examples |
| Verification | Exercise normal, malformed, unauthorized and failure paths | Regression, static-analysis and integration evidence |
| Maintenance | Review dependencies, local signatures and known limitations | Updated audit record and change-driven regression checks |

## Verification approach and current result

Automated cases cover authentication, session expiry/revocation, CSRF, file validation, document reconstruction, metadata handling, parser bounds, YARA-X behavior, scanner errors, DLP restrictions, encryption, ownership, file integrity and concurrent share redemption. The local suite most recently completed 93 tests with no failures. Bandit reported no medium/high findings and seven low findings, primarily due to bounded subprocess calls and an XML output serializer import. The full dependency audit found no known advisories in the resolved dependency graph at audit time. Native Tesseract was not installed on the development workstation, so OCR-to-DLP is tested with a controlled OCR response, not a native OCR acceptance run.

These results apply to the tested source version and cases. They do not establish that the product is unhackable or certified. A ClamAV EICAR fixture returned an error and is recorded as inconclusive rather than a passed detection.

## Feasibility and limitations

The project is technically feasible as a small Python application with local libraries and no mandatory paid scanning service. Test inputs are bounded and safe synthetic examples are provided. Operational correctness depends on preserving private data and keys, keeping ClamAV signatures current, installing Tesseract for OCR and using non-sensitive demonstration files.

Known limitations include the server’s access to encryption keys, no account recovery, in-memory single-process rate limiting, pattern-based DLP, one antivirus engine, limited local YARA-X rules, no dynamic sandbox, possible OCR misses, and no independent penetration test. An HMAC chain detects edits to retained history but cannot prove that a final suffix was not removed without an external checkpoint. These are explicit future-improvement areas, not hidden capabilities.

## References

1. OWASP, File Upload Cheat Sheet.

2. OWASP, Password Storage Cheat Sheet.

3. NIST, SP 800-218: Secure Software Development Framework.

4. Python Cryptographic Authority, Fernet documentation.

5. YARA-X, Python API documentation.

6. Tesseract, documentation.

The research notes explain how selected sources relate to design choices. Replace academic placeholders with genuine institutional details before submission.